

Concord — User Manual

Concord is a personal archive and research tool for video content. It downloads videos (from YouTube or local files), transcribes them with speaker labels, lets you take time-anchored notes, link those notes across the whole archive on a visual map, and chat with everything you've collected using a local LLM.

Everything stays on your machine. No accounts, no cloud sync, no telemetry.

Contents

- [Install](#)
- [First-run setup](#)
- [The main pages](#)
- [Common workflows](#)
- [Voice notes](#)
- [Settings reference](#)
- [Troubleshooting](#)
- [Where your data lives](#)

Install

Mac: Download `Concord.dmg`, drag to Applications, launch. The first time you use voice recording, macOS prompts for microphone access — allow it in System Settings → Privacy & Security → Microphone.

Linux: Download the `.AppImage` (any distro) or `.deb` (Debian/Ubuntu) and run. If the app crashes on launch with a sandbox error, see [Troubleshooting](#).

The app bundles ffmpeg, yt-dlp, and (on Mac) the FluidAudio transcription engine — you don't install them separately.

First-run setup

These run once when you first open the app. You can revisit any of them later from **Settings**.

1. Transcription engine

This is what turns video audio into searchable text. Concord uses different engines depending on your platform:

- **Mac:** FluidAudio on the Apple Neural Engine — bundled, no setup. The first transcription downloads the model (~600 MB) one time.
- **Linux with an NVIDIA GPU:** First launch redirects you to a setup wizard at `/setup/transcription`. It installs the Parakeet engine in a managed Python environment (~5 minutes, no Terminal needed).
- **Linux without a GPU:** Whisper-CPU works but is slow. A GPU machine is strongly recommended for archive-scale work.

You can re-run the wizard or verify the install in **Settings** → **Transcription Engine**.

2. LLM (for chat, summaries, semantic search)

Many of Concord's smarter features rely on a local LLM. The expected setup is [oMLX](#) or any OpenAI-compatible local server on `localhost`.

In **Settings**, set:

- **Base URL:** e.g. `http://localhost:8000/v1`
- **Chat model:** pick from the dropdown after the server is up
- **Embedding model:** a 1024-dimension model like `Qwen3-Embedding-0.6B` (Concord hard-codes 1024 dims for storage consistency)

The status dot in the top bar turns green when the LLM is reachable. The archive, search, transcription, downloading, and notes all work without an LLM — you just won't get chat, AI summaries, or semantic search.

3. YouTube cookies (optional)

Needed for age-restricted, members-only, or private (shared-with-you) videos. In **Settings** → **Pipeline**, either:

- Pick a browser to pull cookies from (`chrome` , `firefox` , `brave` , etc.) — easiest, no manual export
- Point to a `cookies.txt` file you exported by hand

Skip this if you're only grabbing public videos.

4. YouTube Data API key (optional)

Needed for the **Discover** and **Watchers** pages, which call YouTube's official search API.

1. Open the [Google Cloud Console](#), create or pick a project.
2. APIs & Services → Library → enable **YouTube Data API v3**.
3. Credentials → Create Credentials → API key.
4. Restrict the key to "YouTube Data API v3" only.
5. Paste it into **Settings** → **YouTube Data API**.

Free quota is 10,000 units/day, ~100 searches.

5. Add channels

Open **Pipeline** → **Add channel**. Two flavors:

- **YouTube channel** — paste the channel URL or `@handle` . Concord polls it on a schedule and queues new uploads.
- **Local folder** — point at a directory of `.mp4` / `.m4a` / `.wav` / `.mkv` files. They're treated like a channel: scanned, transcribed, searchable.

Each channel has toggles for **Enabled** (pause monitoring without losing config), **Diarization** (speaker labeling — turn off for single-speaker content to save time), and **Include Shorts** (off by default; most Shorts duplicate full videos).

The main pages

The top nav is grouped into three sections:

Content

Library — every video Concord knows about. Filter by channel, status, transcript availability, or starred-only. Sort by upload date, words, title, or recent activity. Click the star at the start of any row to mark it as crucial — the **Starred** chip in the filter row then narrows the list to just those.

Click a row to open the video drawer, which plays the video, shows the transcript with speaker labels, and lets you take notes on highlighted quotes.

Discover — type a phrase, get YouTube search results. The **Include / Exclude** inputs tint matching cards green (matches an include phrase) or red (matches an exclude phrase). They don't hide anything — it's a calibration view so you can see what your filter *would* catch before turning it into a saved Watcher. When include phrases are set, the server scans up to 5 result pages until it has 50 title-matching hits.

Click **Save as watcher** to keep checking for new matches forever.

Watchers — saved searches that poll YouTube on a configurable cadence (default 24 h). Each watcher has:

- **Phrase variants** — titles must literally contain one of these
- **Allowed / Blocked channels** (optional)
- **Enabled** toggle
- **Auto-queue** toggle — when on, new hits go straight to the download queue; when off, they land in the Inbox for manual review
- **Poll interval** (hours)

The Inbox at the top of the page shows everything currently awaiting review — click **Queue** or **Dismiss** on each.

Transcripts — full-text and semantic search across every transcript. Type a phrase and get matching segments with their video,

timestamp, and speaker. Semantic search uses the embedding model and finds passages with similar meaning even when wording differs.

Notes — clip-level notes you've taken. Each note can have multiple *anchors*, one per appearance across different videos at different timestamps. Tag notes for grouping; link them together in the Map.

Analysis

Speakers — every unique voice Concord has detected. Each video's per-speaker voiceprint is matched against existing global speakers when a new transcription finishes. Unidentified speakers show as `s0` , `s1` , etc. — click one to assign a name (e.g. "John Smith"). Future videos featuring the same voice auto-label.

Map — graph view of your notes. Four layout modes:

- **Videos** — each node is a video container with clip rows inside. Good for "show me where these clips live".
- **Cards** — each note is its own card. Good for cluster work.
- **Arc** — vertical timeline with arcs connecting linked notes.
- **Cluster** — physics-driven force layout.

Drag from any clip's handle (left / right / top / bottom) to any handle on another clip to create a manual link. Pick a link type first — each is color-coded:

- **Same topic** (blue)
- **Same claim** (emerald)
- **Contradicts** (red)
- **Follow-up** (violet)
- **Context** (amber)

Two anchors of the same note can each carry their own link to the same target (1:N, N:1, N:N supported). Click an existing link to change its type, or delete it.

AI — chat with your archive. Ask questions; answers come back with citations like `[1]` , `[2]` that point at the specific video

segments used. Conversations are saved automatically; pinned ones sort to the top.

Ops

Pipeline — the operations dashboard. Real-time view of current download and transcription jobs. From here you can:

- Add, edit, enable/disable channels
- Manually download a one-off URL
- Retranscribe a finished video (e.g. after switching engines)
- Import a folder of existing files into a channel
- Update yt-dlp when YouTube changes break it

Status — high-level system info: last channel check, next check, daily download cap, jobs in flight, recent errors.

Common workflows

Archive a YouTube channel

1. **Pipeline** → **Add channel**, paste the channel URL.
2. Wait. Concord scans on the next interval (default 60 min), downloads new uploads, and transcribes them.
3. Browse them in **Library** as they finish.

To grab the channel's existing back catalogue, hit the **Archive** button on the channel card.

Grab one video right now

1. **Pipeline** → **One-off download**, paste the URL.
2. Watch progress in the same page.
3. When done, the row lands in **Library**. If the video belongs to a channel you've already configured, it attaches to that channel automatically (matched first by YouTube ID, then by name).

Import existing files

1. **Pipeline** → either add a new channel pointing at the folder, or use **Import folder** on an existing channel.
2. Concord scans the folder; matching files get pulled into the queue and transcribed.

Take research notes on a video

1. Open the video from **Library** or **Pipeline**.
2. Read or skim the transcript. Highlight a quote — a Notes button appears.
3. Add tags and an optional commentary note.
4. The clip shows up on the **Map** and in **Notes**.

To anchor the same note to another appearance of the same idea, open another video and add a new anchor.

Find connections across notes

1. Open **Map**.
2. Pick a layout — Cards or Videos for hand-curated work, Cluster or Arc for spotting patterns.
3. Drag from any clip handle to any other clip handle to create a manual link. Pick the type first (color-coded).
4. Click an existing link to change its type or delete.

Ask the archive a question

1. **AI** → type the question.
2. Answer arrives with citations. The [N] tags link back to the exact video segments the answer was grounded on.
3. Pin conversations you want to keep at the top.

Watch for new content with a specific person

1. **Discover** → type something like "interview with Jane Doe" .
2. Use Include phrases (with , joins , featuring) and Exclude phrases (reacting to , breakdown , responds to) to tint results.

3. When the phrase combo looks right, **Save as watcher**.
4. The watcher polls on its schedule. Hits land in the Watchers Inbox (or straight in the download queue if auto-queue is on).

Voice notes

The mic button in the top bar opens a recorder. Speak; Concord saves the raw PCM audio and (if a transcription engine is installed) runs it through the same pipeline as a downloaded video. Useful for capturing observations during research without leaving the app.

Settings reference

- **LLM** — base URL, API key, chat + embedding models.
- **Embeddings** — backfill missing embeddings, repair stale ones.
- **AI summaries** — backfill 2–3 sentence per-video summaries.
Without **Overwrite**, skips videos already summarized by the *current* model (so switching models is a re-summarize trigger).
- **Pipeline** — save folders, video quality / codec, YouTube cookies, daily download cap, LAN access (let other devices on your network reach the web UI), transcription model.
- **YouTube Data API** — your Google Cloud key, write-only on the wire (the server returns a masked preview instead of echoing the full secret).
- **Transcription Engine** — read-only summary of the wizard-installed engine + link to re-run the wizard. The model dropdowns elsewhere filter to whatever's actually installed.
- **Library Maintenance** — orphan repair, transcript reindex, vector index reset.

Troubleshooting

"spawn ./venv/bin/python ENOENT" — the selected transcription model doesn't match the engine the wizard installed. Settings now auto-heals this on boot (Parakeet wizard installed + Whisper model saved → migrates to Parakeet's default). If it still happens, re-run the transcription wizard.

Linux: app crashes with a sandbox error — Electron's `chrome-sandbox` needs `setuid root` after install:

```
sudo chown root:root /opt/Concord/chrome-sandbox
sudo chmod 4755 /opt/Concord/chrome-sandbox
```

Never add `--no-sandbox` — it disables the browser sandbox entirely.

yt-dlp errors / downloads suddenly broken — YouTube changes the site every few weeks. **Pipeline** → **Update yt-dlp** pulls the latest release. On Mac the updated binary lives in `~/.concord/bin/yt-dlp` (outside the read-only app bundle, so it survives codesigning).

LLM offline — check the status dot in the top bar. If oMLX isn't running, start it. Chat / summary / semantic-search features go unavailable until it's back; everything else (browse, search, transcribe, download, take notes, map) still works.

Manual download shows under the channel name instead of the configured channel — fixed for new downloads (matched by YouTube ID first, then case-insensitive name). For older orphan rows, rename them from the Pipeline page's virtual-channel rename UI.

A 30-second chunk has two voices in it — chunks are split at speaker turn boundaries on new transcriptions. Retranscribe old videos to apply the split.

Diarization is wasting time on single-speaker channels — open the channel's row in Pipeline and turn off **Diarize**.

Where your data lives

- **Mac:** ~/Library/Application Support/Concord/pipeline.db for the database, plus videos and transcripts in the folders you set under Settings → Pipeline.
- **Linux:** ~/.local/share/concord/pipeline.db plus your configured save folders.
- **Settings** are stored in the database's `app_config` table.
- **YouTube cookies** are referenced by path — Concord never copies the cookie file.

Nothing is uploaded. The LLM is local. The YouTube API key only talks to Google (and only with your key). Everything else stays on disk.